

```

•   __programObject = glCreateProgram();
•   glAttachShader(__programObject, fragShader);
•   glAttachShader(__programObject, vertShader);
•   glLinkProgram(__programObject);
•   // Determine if the shader object is linked to the
program
•   glGetProgramiv(__programObject, GL_LINK_STATUS,
&linked);

```

- d. Get the location for each vertex or pixel attribute in the shader program using the `glGetAttribLocation()` and `glGetUniformLocation()` methods. If the shader program attributes (a position, a color, and `u_mvpMatrix`) are valid, these methods return an integer value that represents the index location of the attributes. For invalid attributes, the return value is -1.

The `glGetAttribLocation()` method retrieves the location of the attributes that have an effect on a vertex or pixel, such as position vector, normal vector, or vertex color. The `glGetUniformLocation()` method retrieves the location of the attributes that have an effect on groups of vertices or pixels, such as model view matrix, projection matrix, or light position.

```

•   __idxPosition = glGetAttribLocation(__programOb-
ject, "a_position");
•   __idxColor = glGetAttribLocation(__programObject,
"a_color");
•   __idxMVP = glGetUniformLocation(__programObject,
"u_mvpMatrix");
•   e. Install the shader program and enable the GPU to
execute the shader operations in the frame buffer.
•   // Install the program object to use the shader pro-
gram
•   glUseProgram(__programObject);
•   f. Finally, the glDeleteShader() method is used to free the
memory and invalidate the name associated with the frag-
ment and vertex shader objects.
•   glDeleteShader(vertShader);
•   glDeleteShader(fragShader);
•
•   // Clear the color buffers
•   glClearColor(0, 0, 0, 1);
•   glClear(GL_COLOR_BUFFER_BIT);

```